Introduction

Container With Most Water shows up constantly in interview prep lists, and there's a good reason for that beyond it just being a popular LeetCode number. It's one of the cleanest examples of a two-pointer pattern that isn't obvious until someone shows it to you — and once it clicks, you start recognizing the same shape in a dozen other array problems.

Interviewers like this one because it splits cleanly into two phases: can you write the correct-but-slow version, and can you reason your way down to something faster? That second part is the real test. It's not really about memorizing "use two pointers here." It's about the ability to look at a brute-force solution, spot exactly what work is wasted, and prove to yourself why it's safe to skip it.

If you're prepping for coding interviews, this problem is worth genuinely understanding rather than memorizing, because the underlying idea — "shrink the search space from both ends, and only move the side that's limiting you" — is a pattern you'll reuse far beyond this one question.

Problem Overview

You're given an array of non-negative integers, where each number represents the height of a vertical line standing on the x-axis. Picture these lines standing side by side on a graph, spaced one unit apart.

Pick any two of these lines. Together with the ground between them, they form a container that can hold water. The height of the water is limited by the shorter of the two lines (water spills over the shorter side), and the width is just the distance between the two indices you picked.

Your task: find the two lines that form the container holding the most water, and return that maximum amount.

One constraint that trips people up: you can't tilt the container. The water level is strictly bounded by `min(height[left], height[right])` — you're not allowed to somehow make the shorter wall "count for more."

Example

Say we have the following heights:

```
index:  0  1  2  3  4  5  6  7  8
height: 1  8  6  2  5  4  8  3  7
```

Nine walls, so there are 36 possible pairs. Somewhere in there, one pair beats all the others.

Take walls at index 1 (height 8) and index 8 (height 7): - Width = 8 - 1 = 7 - Height = min(8, 7) = 7 - Area = 7 × 7 = **49**

That happens to be the best pair for this array — the answer is 49. Every other pair, no matter how tall one of the walls is, produces a smaller number once you account for both the shorter wall and the narrower gap.

Intuition

Before jumping into any algorithm, it helps to picture the problem physically. You've got a row of walls. Pick two, and water pools between them up to the height of the shorter one — anything above that spills over the short side.

The naive instinct is: "just check every pair and keep the best one." That works, and it's worth saying out loud in an interview, but it doesn't scale.

The insight that unlocks the fast solution is about what happens when you *move* a pointer. Start with the widest possible container — the two outer walls. Now ask: if I want to consider a narrower container, which wall should I give up?

If you move the **taller** wall inward, two things can happen: the width shrinks (guaranteed), and the height is now bounded by whichever wall is still shorter — which is still the same short wall as before. So moving the taller wall can only shrink or maintain the area contribution of the height, while definitely shrinking the width. Net result: the area can never improve.

If you move the **shorter** wall inward, the width still shrinks, but now there's a *chance* the new wall is taller than the old short one — which could raise the height cap and offset the narrower width.

So moving the taller wall is a wasted move — it can never lead to a better answer. Moving the shorter wall is the only move that has a chance of improving things. That one observation is the entire algorithm.

Brute Force Solution

Idea: Check every possible pair of walls, compute the area for each, and track the maximum.

Algorithm: 1. For every pair of indices `(i, j)` where `i < j`, compute `min(height[i], height[j]) * (j - i)`. 2. Keep a running maximum. 3. Return the maximum after checking all pairs.

Advantages: - Trivial to reason about and prove correct. - No tricky edge cases to worry about — it just checks everything.

Disadvantages: - Roughly n^2 comparisons. For an array of 100,000 walls, that's around 10 billion checks — far too slow for any reasonable time limit.

Time complexity: $O(n^2)$ **Space complexity:** $O(1)$

Optimal Solution

The two-pointer approach uses the insight above directly: start as wide as possible, and only ever narrow the container from the side that's currently the bottleneck.

Setup: - `left` pointer starts at index 0. - `right` pointer starts at the last index. - `max_area` starts at 0.

Loop, while `left < right`: 1. Compute `width = right - left`. 2. Compute `current_height = min(height[left], height[right])`. 3. Compute `area = width * current_height`, and update `max_area` if it's bigger. 4. Move the pointer at the **shorter** wall one step inward. (If both are equal, move either — conventionally `left`.)

Each iteration either moves `left` forward or `right` backward, never both, and the loop ends when they meet. That guarantees the pointers together cover the array exactly once.

A quick trace on `[1, 8, 6, 2, 5, 4, 8, 3, 7]`:

left	right	width	min height	area	max so far
0 (1)	8 (7)	8	1	8	8
1 (8)	8 (7)	7	7	49	49

The left wall was shorter (height 1), so it moved inward first. That single move already produces the winning pair. The pointers keep closing in after this, but nothing beats 49 for this array.

Python Code

```
def max_area(height: list[int]) -> int:
    """Return the maximum water a container formed by two walls can hold."""
```

```
    left, right = 0, len(height) - 1
    max_area = 0

    while left < right:
        width = right - left
        current_height = min(height[left], height[right])
        max_area = max(max_area, width * current_height)

        # The shorter wall is the bottleneck, so only it is worth moving.
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return max_area
```

Code Walkthrough

- `left, right = 0, len(height) - 1` : pointers start at the two ends, giving the widest possible container first.
- `max_area = 0` : safe starting value since area can never be negative.
- `while left < right` : the loop stops the moment the pointers meet — no container exists with zero width.
- `width = right - left` : distance between the two chosen walls.
- `current_height = min(height[left], height[right])` : water can't rise past the shorter wall, so this is the actual usable height, not the sum or the average.
- `max_area = max(max_area, width * current_height)` : update the best answer seen so far, without needing an explicit `if` statement.
- `if height[left] < height[right]: left += 1 else: right -= 1` : the core of the algorithm — always discard the shorter wall's current position, since keeping it can never beat what's already been computed.

Dry Run

Using `[1, 8, 6, 2, 5, 4, 8, 3, 7]`:

1. `left=0, right=8` → width=8, min(1,7)=1 → area=8 → max_area=8. Left is shorter → `left=1`.
2. `left=1, right=8` → width=7, min(8,7)=7 → area=49 → max_area=49. Right is shorter → `right=7`.
3. `left=1, right=7` → width=6, min(8,3)=3 → area=18. Right is shorter → `right=6`.

4. `left=1, right=6` → width=5, min(8,8)=8 → area=40. Equal, move left → `left=2`.
5. …pointers keep closing, but nothing exceeds 49.

Final answer: **49**, matching the expected result.

Complexity Analysis

Time: O(n). Each iteration advances exactly one pointer inward, and the loop ends when they meet. Across the whole run, `left` and `right` together take at most `n` total steps, so the array is effectively traversed once.

Space: O(1). Only a fixed handful of variables (`left`, `right`, `max_area`, and loop temporaries) are used, regardless of input size — no auxiliary arrays or recursion stack.

This is correct, not just fast, because of the proof from the intuition section: moving the taller wall inward can never produce a better area than what's already been captured, so it's always safe to discard it without checking those pairs explicitly.

Alternative Solutions

There isn't a meaningfully different approach that beats O(n) here — this is already optimal, since you must look at the array at least once to know the heights. Some engineers reach for a stack-based or monotonic-stack approach out of habit (it's useful for the related **Trapping Rain Water** problem), but for *this* problem it adds complexity without any benefit — the two-pointer approach is both simpler and already linear.

Edge Cases

- **Two walls only:** the array has exactly one possible pair, which is trivially the answer — no search needed.
- **All equal heights:** `min()` always returns the shared height, and the widest pair (the two ends) will be optimal.
- **A zero-height wall:** contributes zero area for any pair that uses it, but doesn't break the loop or the math.
- **Strictly increasing or decreasing heights:** the algorithm still works, since it never assumes any particular shape — it only compares the two current pointers.
- **Empty or single-element input:** technically undefined per problem constraints (LeetCode guarantees at least 2 elements), but the loop condition `left < right` protects against an index

error if it ever happens.

Common Mistakes

1. **Adding the two heights instead of taking the minimum.** Water can't rise above the shorter wall, so it's `min()`, not `sum()` — this is the single most common bug on this problem.
2. **Moving the taller pointer instead of the shorter one.** This breaks the core proof of correctness and can skip over the actual best answer.
3. **Using `<=` in the loop condition.** Since `left == right` means zero width, the loop should stop before that point, not process it.
4. **Forgetting to update `max_area` before moving a pointer.** The area must be computed for the *current* pair before either pointer moves.
5. **Off-by-one errors initializing `right`.** It should start at `len(height) - 1`, not `len(height)`.

Interview Questions

- Why is it safe to discard the shorter wall's position instead of checking all pairs that use it?
- How would you modify this to also return the indices of the winning pair, not just the area?
- What changes if negative heights were allowed (they aren't here, but it's a good sanity check for your understanding)?
- Can you solve this recursively, and what would the space complexity become?
- How does this problem relate to Trapping Rain Water, and why does that one need a different approach?

Similar Problems

- **Trapping Rain Water (LeetCode 42):** Same array of wall heights, but now every wall matters — you're summing trapped water across the whole terrain, not picking just two walls.
- **Two Sum II - Input Array Is Sorted:** Another classic two-pointer pattern, closing in from both ends based on a comparison.
- **3Sum:** Uses two pointers as an inner loop after fixing one element, building directly on this pattern.
- **Squares of a Sorted Array:** Two pointers from both ends, comparing absolute values instead of heights.

Key Takeaways

The brute force is easy to write and easy to trust, but n² doesn't survive contact with large inputs. The breakthrough is realizing that moving the *taller* wall can never help — it only shrinks width while keeping (or worsening) the height cap. That one observation converts a 10-billion-check problem into a 100,000-step one. Whenever you see two pointers closing in from opposite ends of an array, ask which side is actually the limiting factor, and only move that one.

Watch the Video

For the full walkthrough with the step-by-step trace on all nine walls, watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series focused on making interview-style problems actually make sense — not just memorizing solutions, but building the intuition to derive them yourself under pressure.

Call To Action

If this helped the two-pointer pattern click, leave a like on the video and subscribe on YouTube for more problems broken down the same way. For deeper write-ups like this one delivered straight to your inbox, subscribe to the Substack. And if something felt off, unclear, or you want to see a specific problem covered next, drop a comment — that's genuinely what shapes what gets covered here.

Quick Review

Which pattern fits shrinking-search over a sorted/array boundary problem like this?

Two-pointer squeeze: start at both ends, move the pointer bounding the smaller side inward.

Time complexity of the two-pointer approach here?

O(n) — single pass, each pointer moves at most n times total.

Space complexity of the two-pointer approach?

O(1) — only a few running variables, no extra structures.

Common bug when moving pointers in this solution?

Moving both pointers at once, or moving the taller side — either can skip the actual best answer.

Brute force vs two-pointer: what's the tradeoff?

Brute force checks all pairs, $O(n^2)$ but obviously correct; two-pointer is O(n) but needs the greedy-move proof.

Interview follow-up: why is it safe to discard the shorter side's pointer?

Width only shrinks going forward, so keeping the shorter line can never beat a taller one at any future width.

Container With Most Water — free Python cheat sheet